

Project 2 — One-Time Pad: Alice's Encryption

This notebook implements Alice's portion of the one-time-pad exercise.

Alice:

1. Composes the plaintext message within the permitted keyspace.
2. Generates a key equal in length to the plaintext.
3. Converts both values to bytes.
4. XORs the plaintext with the key to compute the ciphertext.
5. Writes the ciphertext to a hexadecimal text file.
6. Writes the key from a hexadecimal text file.

```
In [1]: from pathlib import Path
import secrets
```

Functions: Post the ciphertext & key

The following two code blocks are the XOR function `xor_bytes()` which contains the exception handler which evaluates if the ciphertext and key are non-matching in length, and the `write_hex_file()` to write out in the ASCII rendered in hexadecimal content.

```
In [2]: def xor_bytes(message: bytes, key: bytes) -> bytes:
        if len(message) != len(key):
            raise ValueError("Message and key must have the same length.")

        return bytes(m ^ k for m, k in zip(message, key))
```

```
In [3]: def write_hex_file(filename: str, data: bytes) -> None:
        Path(filename).write_text(data.hex(), encoding="ASCII")
```

Alice's Encrypt & Write

This section will prompt Alice to input the plain txt message which is then processed using the `token_bytes()` from the **secrets** library to generate a streamed key matching the plaintext character for character. The key is then XOR'ed against the plaintext message to render the ciphertext with the one-time pad.

Outputs & Proofs

- Alice's prompt and response

- Alice's plaintext
- Plaintext in hexadecimal
- Key in hexadecimal --> output as key.txt
- Ciphertext in hexadecimal --> output as ciphertext.txt

```
In [4]: plaintext = input("Alice, enter the plaintext message: ")

plaintext_bytes = plaintext.encode("ascii")

# Generate one random key byte for every plaintext byte
key = secrets.token_bytes(len(plaintext_bytes))

# OTP encryption: C = M XOR K
ciphertext = xor_bytes(plaintext_bytes, key)

print("Plaintext:      ", plaintext)
print("Plaintext hex:  ", plaintext_bytes.hex())
print("Key hex:         ", key.hex())
print("Ciphertext hex: ", ciphertext.hex())

write_hex_file("ciphertext.txt", ciphertext)
write_hex_file("key.txt", key)

print("\nSaved ciphertext.txt and key.txt")
```

```
Plaintext:      My, oh my what a fine day.
Plaintext hex:  4d792c206f68206d79207768617420612066696e65206461792e
Key hex:        05f7bc9fa69bbaf6ff561aeda3e2ee57356493fbf826a41a7c9b
Ciphertext hex: 488e90bfc9f39a9b86766d85c296ce361502fa959d06c07b05b5
```

Saved ciphertext.txt and key.txt

In []: